UNIVERSIDADE DO OESTE DE SANTA CATARINA
ÁREA DE CIÊNCIAS EXATAS E TECNOLÓGICAS

Curso - Engenharia de Computação
29 de Outubro de 2026 Componente - Inteligência Artificial e Sistemas Inteligentes

---

SPRINT 4

ANÁLISE DOS RESULTADOS

Aluna: Ana Clara Viganó P. de Oliveira - 412887 **Aluna:** Jasmine Masson Alves - 427466

---

1 Introdução

Integrar os transformer em uma arquitetura GPT simplificada consiste em conectar vários blocos transformer em sequência, com mecanismos de atenção, normalização, redes FeedForward e conexões residuais. Isso permite que o modelo processe os tokens de entrada, construa representações contextualizadas do texto e produza logits para os possíveis próximos tokens. Já a inferência e a geração de texto é o processo em que o modelo utiliza uma sequência inicial de tokens para prever o próximo token. A partir dos logits, são calculadas probabilidades para os tokens possíveis, e um deles é adicionado à sequência. Tal processo se repete até atingir a quantidade de tokens necessárias o que permite a geração de texto de forma autoregressiva.

A implementação, desses conceitos, foi desenvolvida em Python com a biblioteca PyTorch. A configuração inicial utiliza os principais hiperparâmetros associados à arquitetura GPT-2 de 124 milhões de parâmetros, vocabulário de 50.257 tokens, dimensão de embedding de 768, 12 cabeças de atenção e 12 blocos Transformer. Nesta etapa, o objetivo é validar a estrutura e o fluxo dos tensores e não treinar um modelo de linguagem completo.

2 Implementação do Transformer Block

O *Transformer Block* é o centro da arquitetura Transformer. Ele processa as representações dos tokens usando dois conceitos principais, o mecanismo de atenção multi-cabeças (*Multi-Head Attention*) e uma rede neural *FeedForward*. Antes de cada componente é aplicada a normalização *LayerNorm*, depois do processamento o resultado é combinado com a entrada original por meio de uma conexão shortcut. O *Dropout* é utilizado para ajudar a reduzir o overfitting durante o treinamento.

Na sprint, essa estrutura está implementada na classe `TransformerBlock`, no arquivo `transformer_block_class.py`. A classe reutiliza a implementação de `MultiHeadAttention` desenvolvida na Sprint 3 e as classes `FeedForward` e `LayerNorm` implementadas nos demais arquivos do projeto.

2.1 Componentes

COMPONENTE	FUNÇÃO
Multi-Head Attention	Permite que cada posição da sequência combine informações relevantes de outras posições, considerando diferentes "heads"de atenção.
LayerNorm	Normaliza os valores da representação de cada token ao longo da dimensão de embedding. Treinamento mais estável.
FeedForward	Aplica transformações lineares e uma função de ativação não linear a cada posição da sequência. Na sprint, a dimensão interna é expandida de 768 para 3072 e depois reduzida para 768.
Conexões shortcut	Somam a entrada de um subcomponente à sua saída, isso ajuda a preservar informação e favorece o fluxo dos gradientes durante o treinamento.
GELU	Função de ativação não linear usada entre as duas camadas lineares da rede FeedForward.
Dropout	Desativa aleatoriamente parte das ativações durante o treinamento para ajudar a reduzir o overfitting. No modo de avaliação fica desativado, ativa-se apenas em treinamento.

Tabela 1 - Componentes que formam o Transformer Block

2.2 Fluxo de processamento

A implementação segue o padrão *Pre-LayerNorm*, ou seja, a entrada é normalizada antes de ser enviada a cada componente. Abaixo segue a descrição do fluxo.

1. A entrada original é guardada em uma variável chamada shortcut.
2. A entrada passa por norm1, pelo mecanismo de atenção e pelo dropout.
3. O resultado é somado a entrada original, formando a primeira conexão residual.
4. O resultado dessa soma é guardado como uma nova entrada residual.
5. A representação passa por norm2, pela rede FeedForward e pelo Dropout.
6. A saída da rede FeedForward é somada à entrada dessa segunda etapa.
7. O tensor resultante é retornado pelo bloco.

As operações residuais podem ser representadas por:

$$x' = x + \text{Dropout}\big(\text{Attention}(\text{LayerNorm}(x))\big)$$

$$y = x' + \text{Dropout}\big(\text{FeedForward}(\text{LayerNorm}(x'))\big)$$

Nessas expressões, x é o tensor de entrada, x' é o resultado após a etapa de atenção e y é a saída do bloco.

Primeiro, o modelo analisa as informações usando a atenção (Attention) e soma o resultado com a entrada original. Depois, passa esse resultado pelo FeedForward e soma novamente com as informações anteriores. Assim, o bloco consegue aprender coisas novas sem perder as informações que já tinha.

2.3 Dimensões dos tensores

O teste realizado no arquivo `transformer_block_class.py` utiliza um tensor com formato `[2, 4, 768]`. As dimensões representam:

- **2**: quantidade de sequências no lote (*batch size*).
- **4**: quantidade de tokens em cada sequência.
- **768**: dimensão do vetor de representação de cada token (*embedding dimension*).

ETAPA	OPERAÇÃO	FORMATO ESPERADO
Entrada	Representações dos tokens	`[2, 4, 768]`
Normalização 1	LayerNorm	`[2, 4, 768]`
Atenção	Multi-Head Attention	`[2, 4, 768]`
Primeira soma residual	Atenção + entrada original	`[2, 4, 768]`
Normalização 2	LayerNorm	`[2, 4, 768]`
Expansão FeedForward	Linear: $768 \rightarrow 3072$	`[2, 4, 3072]`
Redução FeedForward	Linear: $3072 \rightarrow 768$	`[2, 4, 768]`
Segunda soma residual	FeedForward + entrada da etapa	`[2, 4, 768]`

Tabela 2 - Dimensões do Transformer Block

Esse comportamento permite empilhar vários blocos em sequência.

2.4 Teste da implementação

O teste do arquivo cria uma entrada aleatória, inicializa o bloco com a configuração GPT e executa o processamento.

torch.manual$_seed(123)$

x = torch.rand(2, 4, 768)

block = TransformerBlock(GPT$_CONFIG_124M$)

output = block(x)

print("Input shape:", x.shape)

print("Output shape:", output.shape)

O teste verifica o formato dos tensores, mas, isoladamente, não comprova a qualidade das previsões nem o aprendizado de padrões de linguagem.

3 Construção do modelo GPT

3.1 Organização da arquitetura

A arquitetura GPT foi implementada na classe `GPTModel`, no arquivo `GPT_model.py`. Ela reúne os componentes necessários para processar uma sequência de IDs de tokens e produzir logits para cada posição da sequência.

1. **Embedding de tokens**: transforma cada ID de token em um vetor de 768 valores.
2. **Embedding posicional**: representa a posição de cada token na sequência, também com 768 valores.
3. **Dropout de embeddings**: aplica regularização durante o treinamento.
4. **Empilhamento de Transformer Blocks**: aplica 12 blocos em sequência, conforme a configuração atual.
5. **LayerNorm final**: normaliza as representações produzidas pelo último bloco.
6. **Camada de saída**: converte cada vetor de 768 valores em 50.257 logits, um para cada token do vocabulário.

3.2 Configuração do modelo

Os valores são definidos no arquivo `cfg.py`, por meio do dicionário `GPT_CONFIG_124M`.

Parâmetro	Valor	Significado
`vocab_size`	50.257	Quantidade de tokens possíveis no vocabulário.
`context_length`	1.024	Comprimento máximo de contexto configurado para o modelo.
`emb_dim`	768	Quantidade de valores que representa cada token.
`n_heads`	12	Número de cabeças de atenção por bloco.
`n_layers`	12	Número de Transformer Blocks empilhados.
`drop_rate`	0,1	Taxa de dropout configurada em 10%.
`qkv_bias`	`False`	Define que as projeções de query, key e value não utilizam viés.

Tabela 3 - Parâmetros definidos

Entretanto, o número efetivo de parâmetros depende dos detalhes da implementação, incluindo a camada de saída e o compartilhamento ou não de pesos entre embeddings e saída. Por isso, a quantidade de parâmetros deve ser confirmada no modelo.

3.3 Fluxo dos dados no modelo

O método `forward` define como os dados percorrem a arquitetura. A entrada é um tensor de IDs de tokens, com formato `[batch_size, seq_len]`. O modelo consulta os embeddings de tokens e de posições e soma suas representações. Depois, o resultado passa pelo dropout, pelos Transformer Blocks, pela normalização e pela camada de saída. O fluxo pode ser entendido como a representação descrita abaixo.

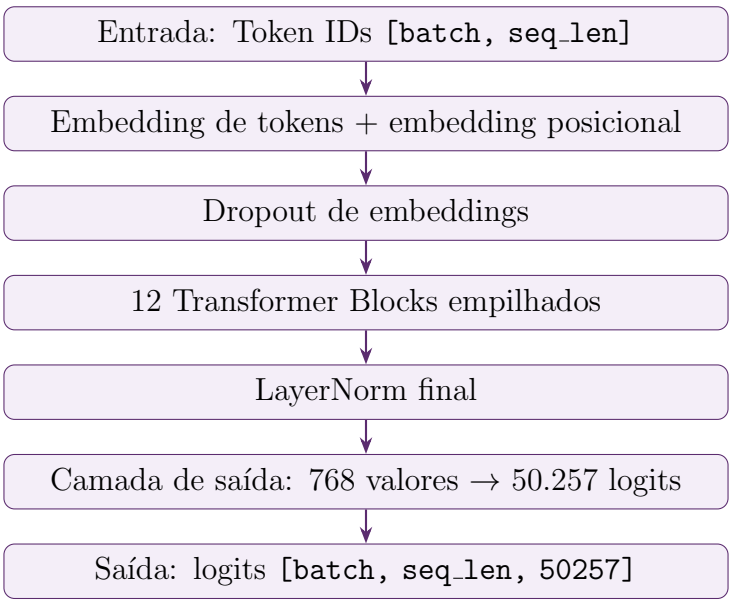


Figura 1 - Fluxo

3.4 Dimensões da entrada e da saída

Para validar o fluxo, o arquivo `data_flow.py` tokeniza duas frases de exemplo: *"Every effort moves you"* e *"Every day holds a"*. Os IDs do tokenizador são reunidos em um lote. Nesse exemplo, cada frase possui quatro tokens, resultando em um tensor de entrada de formato `[2, 4]`.

ESTÁGIO	FORMATO	DESCRIÇÃO
Entrada tokenizada	`[2, 4]`	Duas sequências com quatro IDs de tokens cada.
Embeddings combinados	`[2, 4, 768]`	Cada token é representado por 768 valores.
Saída dos Transformer Blocks	`[2, 4, 768]`	Representações contextualizadas dos tokens.
Logits finais	`[2, 4, 50257]`	Pontuações para todos os tokens do vocabulário em cada posição.

Tabela 4 - Formatos dos tensores

3.5 Logits

Os logits são pontuações numéricas não normalizadas produzidas pela camada de saída. Um valor maior indica uma pontuação relativamente maior para aquele token, mas os logits não são, por si só, probabilidades. Para convertê-los em uma distribuição de probabilidades, pode-se aplicar a função `softmax`.

Na geração autoregressiva, o código seleciona os logits da última posição da sequência e escolhe o token com maior probabilidade por meio de `argmax`, esse procedimento é conhecido como *greedy decoding*. A geração está presente no arquivo `GPT_model.py`.

3.6 Teste de parâmetros

Com a entrada de formato `[2, 4]`, a saída esperada é `[2, 4, 50257]`. O código também calcula o número de parâmetros registrados no modelo. Esse cálculo é importante porque o total depende da implementação. A camada de embeddings de tokens e a camada de saída são declaradas separadamente, sem compartilhamento de pesos (*weight tying*). Assim, a contagem real impressa pelo programa é o resultado da implementação e não garante exatamente 124 milhões de parâmetros.

4 Inferência e Geração de Texto

O processo de geração de texto em um modelo GPT funciona de maneira autorregressiva Isso significa que o modelo escreve como se estivesse completando uma frase palavra por palavra (ou token por token), ele lê o que tem até o momento, adivinha qual é o próximo token, adiciona esse token ao final da frase e usa tudo isso como base para adivinhar o próximo e assim por diante.

4.1 Função de geração

O código implementado para realizar essa geração segue um ciclo dividido em três passos:

1. **Ajuste do Contexto:** O modelo tem um limite de quantas palavras consegue olhar de uma vez (*context_size*). Se a frase for muito longa, a função seleciona apenas os últimos tokens relevantes para não estourar o limite.

$$\text{idx_cond} = \text{idx}[\text{context_size}]$$

2. **Análise do Modelo (*Forward*):** Colocamos o modelo em modo de avaliação (`model.eval()`) com o cálculo de gradientes desligado. Os tokens entram no GPT e percorrem todas as camadas de embedding e blocos transformers.

3. **Foco na Última Palavra:** Como o objetivo é descobrir *o que vem a seguir*, o modelo descarta o passado e olha apenas para as pontuações referentes à última posição da sequência:

$$\text{logits} = \text{logits}[-1]$$

4.2 De Logits a Palavras

Para transformar os números internos da rede neural em texto legível, o modelo passa por um "funil"de três etapas:

- **Logits:** São pontuações brutas que o modelo atribui a cada uma das 50.257 palavras possíveis do vocabulário. Quanto maior o valor do logit para uma palavra, mais o modelo "prefere"ela para aquela posição.
- **Softmax:** Como os logits são números soltos, aplicamos a função Softmax para transformá-los em porcentagens/probabilidades (valores entre 0 e 1 que somam 100%).
- **Seleção do Token (`torch.argmax`):** Na abordagem mais simples implementada, o modelo escolhe de forma "gulosa"a palavra que obteve a maior probabilidade de todas.

4.3 Ciclo e Comportamento

Após descobrir o ID do próximo token, ele é colado no final da sequência original usando a operação de concatenação (`torch.cat`). Esse laço repete-se até atingir o limite desejado de novos tokens (`max_new_tokens`).

Vale destacar que, como o modelo utilizado nesta etapa ainda possui seus pesos gerados de forma aleatória (ou seja, **ele ainda não passou pelo treinamento** com textos reais), as saídas geradas aparecem como palavras sem sentido ou desconexas. O grande objetivo desta entrega foi validar que o fluxo de dados do modelo consiga receber uma frase, processá-la pelas camadas e retroalimentar-se corretamente para gerar novos elementos de forma autônoma.

5 Experimentos

Para avaliar o comportamento da arquitetura e o impacto dos hiperparâmetros na eficiência computacional do modelo, foi realizado 4 experimentos, onde foram analisadas variações no número de blocos transformers, dimensões dos embeddings, comprimento da geração de texto e taxas de dropout.

5.1 Experimento 1: Quantidade de Blocos Transformer

Neste teste, varia-se o número de camadas (`n_layers`) do modelo para 2, 4 e 6, mantendo os demais parâmetros da configuração base do GPT. O objetivo foi observar como o empilhamento de blocos afeta o total de parâmetros e o tempo de execução durante a inferência.

- **Comportamento Observado:** Conforme o número de blocos aumenta, a quantidade total de parâmetros e o tempo de processamento escalam de forma proporcional, visto que cada bloco adicional replica estruturas complexas de atenção e *FeedForward*.
- **Custo Computacional:** Modelos com menos blocos (ex: 2 blocos) executam significativamente mais rápido e consomem menos recursos de hardware.

5.2 Experimento 2: Dimensão dos Embeddings

Avalia-se o impacto da alteração da dimensão do embedding testando os valores 192, 384 e 768 (fixo em 2 camadas).

- **Restrição Arquitetural:** Foi imposta a validação de que a dimensão escolhida deve ser estritamente divisível pelo número de cabeças de atenção (`n_heads` = 12), garantindo que os vetores possam ser fatiados corretamente no mecanismo de *Multi-Head Attention*.
- **Impacto:** Valores maiores de `emb_dim` ampliam drasticamente a capacidade representativa da rede (aumentando o número de parâmetros na camada de embedding e nas projeções lineares), o que aumenta o custo computacional e o tempo de resposta do modelo.

5.3 Experimento 3: Geração de Tokens

Utilizando um modelo reduzido (2 blocos) e a frase inicial *"Muitos divos e divas aprendem IA"*, medimos o tempo necessário para gerar 5, 10 e 20 novos tokens de forma autorregressiva.

- **Análise:** Como a geração autorregressiva exige que o modelo passe por todas as camadas repetidas vezes para cada novo token gerado (passo a passo), o tempo de execução cresce de forma linear com a quantidade de tokens solicitada.
- **Resultado Textual:** As saídas geradas refletiram a natureza estocástica/aleatória de um modelo não treinado, confirmando que a engrenagem de retroalimentação do contexto (*context_size*) e a concatenação (`torch.cat`) operam de maneira íntegra e sem vazamentos de memória.

5.4 Experimento 4: Taxa de dropout

Testa-se o comportamento da rede ao aplicar diferentes taxas de *dropout* (0.0, 0.1 e 0.3) comparando as saídas obtidas no **modo de treinamento** (`model.train()`, onde o dropout desliga conexões aleatoriamente) versus o **modo de avaliação** (`model.eval()`, onde todas as conexões permanecem ativas).

- **Resultados da Diferença Média:**
  - Para `drop_rate = 0.0`, a diferença média entre os modos treino e avaliação foi exatamente **0.000000**, pois o dropout desativado não altera os pesos em nenhum dos cenários.
  - Para `drop_rate = 0.1` e `0.3`, observou-se uma divergência numérica proporcional na saída da rede, evidenciando a aleatoriedade introduzida pelo mecanismo para prevenir o *overfitting* durante a fase de aprendizado.

6 Conclusão

Os códigos desenvolvidos implementam os dois componentes previstos nos Entregáveis 2 e 3. O Transformer Block integra atenção multi-cabeças, normalização, FeedForward, GELU, dropout e conexões residuais, preservando o formato do tensor de entrada. O GPT organiza embeddings de tokens e posições, múltiplos blocos Transformer, normalização final e uma camada de saída que gera logits para o vocabulário.